МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федерального государственного бюджетного образовательного учреждения высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Техникум Пермского института (филиала)

Сопроводительная записка по дисциплине

«Поддержка и тестирование программных модулей»

по теме:

«Создание калькулятора»

Руководитель

/Берестов Д.Б./

« » 2026

г. Исполнитель

/Кушнин Е.А./

« » 2026 г.

Пермь, 2026 г.

ВВЕДЕНИЕ 2
ПОСТАНОВКА ЗАДАЧИ 3
АРХИТЕКТУРА РЕШЕНИЯ 3
РЕАЛИЗАЦИЯ БИБЛИОТЕКИ КЛАССОВ 5
МОДУЛЬНОЕ ТЕСТИРОВАНИЕ 6
РЕАЛИЗАЦИЯ WPF-ИНТЕРФЕЙСА 7
РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ 8
ЗАКЛЮЧЕНИЕ 9

ВВЕДЕНИЕ

В данной работе рассматривается процесс разработки десктопного приложения «Калькулятор» с использованием технологии Windows Presentation Foundation (WPF) в среде Microsoft Visual Studio. Целью данной работы является создание функционального калькулятора, выполняющего базовые арифметические операции, с четким разделением бизнес-логики и пользовательского интерфейса, а также покрытием кода модульными тестами.

Для достижения поставленной цели необходимо решить ряд задач, среди которых разработка библиотеки классов с логикой вычислений, создание WPF-приложения для отрисовки интерфейса, написание модульных тестов для проверки корректности работы и документирование всего процесса разработки. Актуальность темы обусловлена тем, что разделение приложения на независимые слои является стандартной и важнейшей практикой в современной промышленной разработке программного обеспечения, что обеспечивает переиспользуемость кода, упрощает его тестирование и дальнейшую поддержку.

ПОСТАНОВКА ЗАДАЧИ

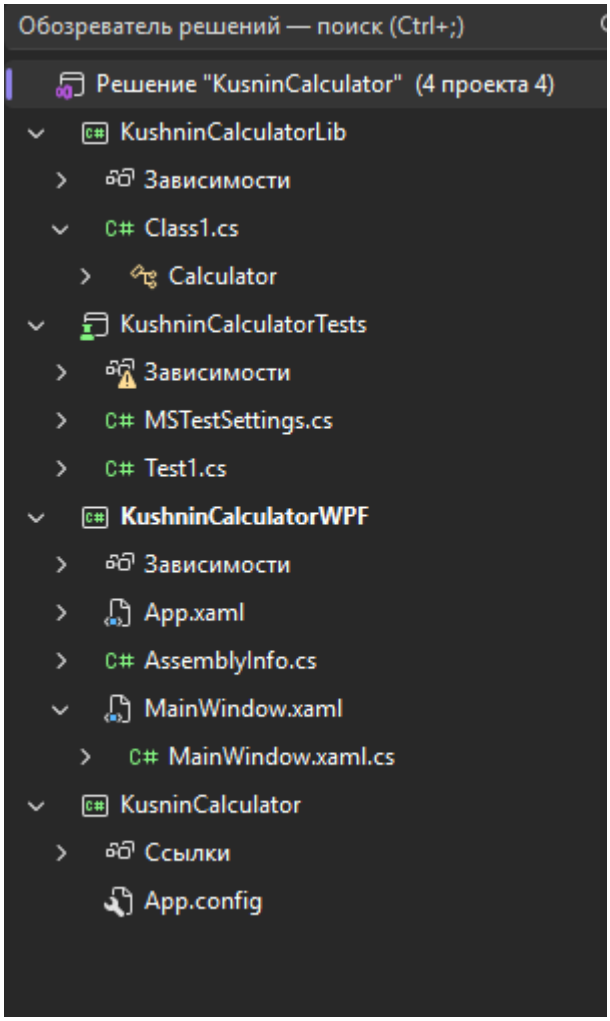
Разрабатываемое приложение должно удовлетворять ряду функциональных и нефункциональных требований. К основным функциональным требованиям относится возможность приема двух числовых операндов от пользователя, выполнение над ними операций сложения, вычитания, умножения, деления и возведения в степень, а также корректная обработка ошибок ввода и попытки деления на ноль с выводом соответствующего сообщения.

Нефункциональные требования включают в себя строгое разделение архитектуры на библиотеку классов и WPF-приложение, а также покрытие тестами не менее восьмидесяти процентов методов библиотеки. В качестве основного инструментария для разработки была выбрана интегрированная среда Microsoft Visual Studio 2022, платформа .NET, фреймворк модульного тестирования MSTest и технология WPF для построения пользовательского интерфейса.

АРХИТЕКТУРА РЕШЕНИЯ

Решение KushninCalculator состоит из трех логически связанных проектов, каждый из которых выполняет свою строго определенную роль. Первым проектом является KushninCalculatorLib, представляющий собой библиотеку классов, в которой инкапсулирована вся математическая логика. Вторым проектом выступает KushninCalculatorTests, содержащий набор модульных тестов для автоматической проверки работоспособности библиотеки. Третьим проектом является KushninCalculatorWPF — непосредственно пользовательское приложение, отвечающее за взаимодействие с конечным пользователем.

Взаимодействие между компонентами строится по принципу разделения ответственности. WPF-приложение не содержит в себе никаких вычислительных алгоритмов, а лишь вызывает методы библиотеки классов, передавая им введенные пользователем данные. Проект тестов также ссылается исключительно на библиотеку классов, что позволяет тестировать бизнес-логику независимо от графического интерфейса.



РЕАЛИЗАЦИЯ БИБЛИОТЕКИ КЛАССОВ

Процесс создания библиотеки классов начался с выбора соответствующего шаблона проекта в среде Visual Studio. Основным и единственным элементом библиотеки является публичный класс Calculator. Данный класс содержит шесть методов, пять из которых реализуют конкретные арифметические операции, а шестой метод является универсальным диспетчером.

Метод Add принимает два числа и возвращает их сумму. Метод Subtract возвращает разность переданных операндов. Метод Multiply реализует операцию умножения. Метод Divide выполняет деление, при этом в его теле реализована явная проверка делителя на равенство нулю: в случае передачи нуля метод генерирует исключение DivideByZeroException с понятным текстовым описанием ошибки. Метод Power использует встроенную функцию Math.Pow для возведения первого

аргумента в степень, заданную вторым аргументом. Метод Calculate принимает два числа и строковое представление оператора, после чего с помощью конструкции switch-case вызывает соответствующий метод вычисления.

```
using System;

namespace KushninCalculatorLib
{
    Ссылок: 4
    public class Calculator
    {
        Ссылок: 3
        public double Add(double a, double b)
        {
            return a + b;
        }

        Ссылок: 2
        public double Subtract(double a, double b)
        {
            return a - b;
        }

        Ссылок: 3
        public double Multiply(double a, double b)
        {
            return a * b;
        }

        Ссылок: 3
        public double Divide(double a, double b)
        {
            if (b == 0)
            {
                throw new DivideByZeroException("Деление на ноль невозможно");
            }
            return a / b;
        }

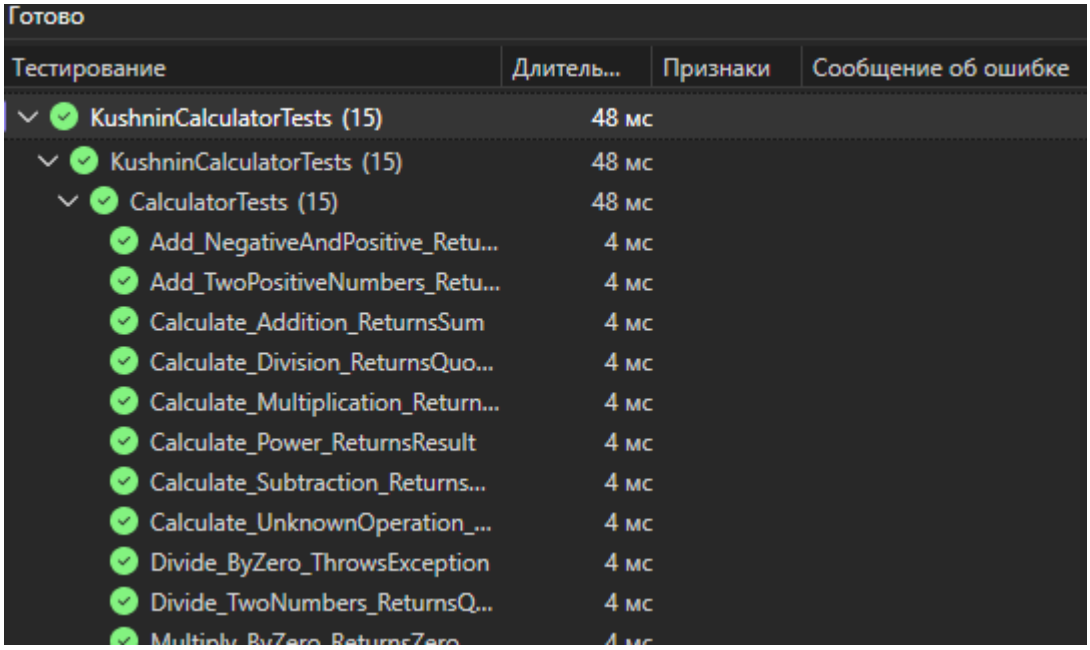
        Ссылок: 3
        public double Power(double baseNumber, double exponent)
```

МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

Для обеспечения надежности и корректности кода был создан отдельный проект модульных тестов KushninCalculatorTests. В классе CalculatorTests был объявлен приватный объект калькулятора, который инициализируется в методе Setup, помеченном атрибутом TestInitialize, что гарантирует создание чистого экземпляра перед запуском каждого теста.

Всего было разработано пятнадцать тестовых методов, покрывающих все основные сценарии использования. Для проверки стандартных операций написаны тесты, передающие положительные и

отрицательные числа и сравнивающие результат с ожидаемым значением с помощью метода Assert.AreEqual. Особое внимание было уделено проверке граничных условий и обработки исключений. Для тестирования деления на ноль и передачи неизвестного оператора использована конструкция try-catch: в блоке try вызывается тестируемый метод, а сразу за ним вызывается Assert.Fail, который принудительно провалит тест, если исключение не будет выброшено. Блок catch перехватывает ожидаемое исключение, что означает успешное прохождение теста.



РЕАЛИЗАЦИЯ WPF-ИНТЕРФЕЙСА

Пользовательский интерфейс приложения реализован с использованием технологии WPF и языка разметки XAML. Окно MainWindow содержит сетку Grid, в которой расположены текстовые метки, поля ввода для первого и второго операндов, поле для ввода символа операции, кнопка вычисления и поле для вывода результата. Также предусмотрено отдельное текстовое поле красного цвета для отображения сообщений об ошибках. Все элементы выровнены по сетке для обеспечения аккуратного внешнего вида.

Обработка событий нажатия кнопки вынесена в файл MainWindow.xaml.cs. В обработчике события BtnCalculate_Click первым

делом очищаются поля результата и ошибки. Далее с помощью метода double.TryParse происходит попытка преобразования введенного текста в числа. Если преобразование не удается, генерируется исключение FormatException. После успешного парсинга вызывается метод Calculate из библиотеки классов, а полученный результат выводится в соответствующее текстовое поле. Вся логика обработки обернута в блок try-catch, который перехватывает любые исключения и выводит их текст в поле для ошибок, предотвращая аварийное завершение работы приложения.



```
<Window x:Class="KushninCalculatorWPF.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="KushninCalculator" Height="350" Width="320"
        WindowStartupLocation="CenterScreen"
        ResizeMode="CanMinimize">
    <Grid Margin="15">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

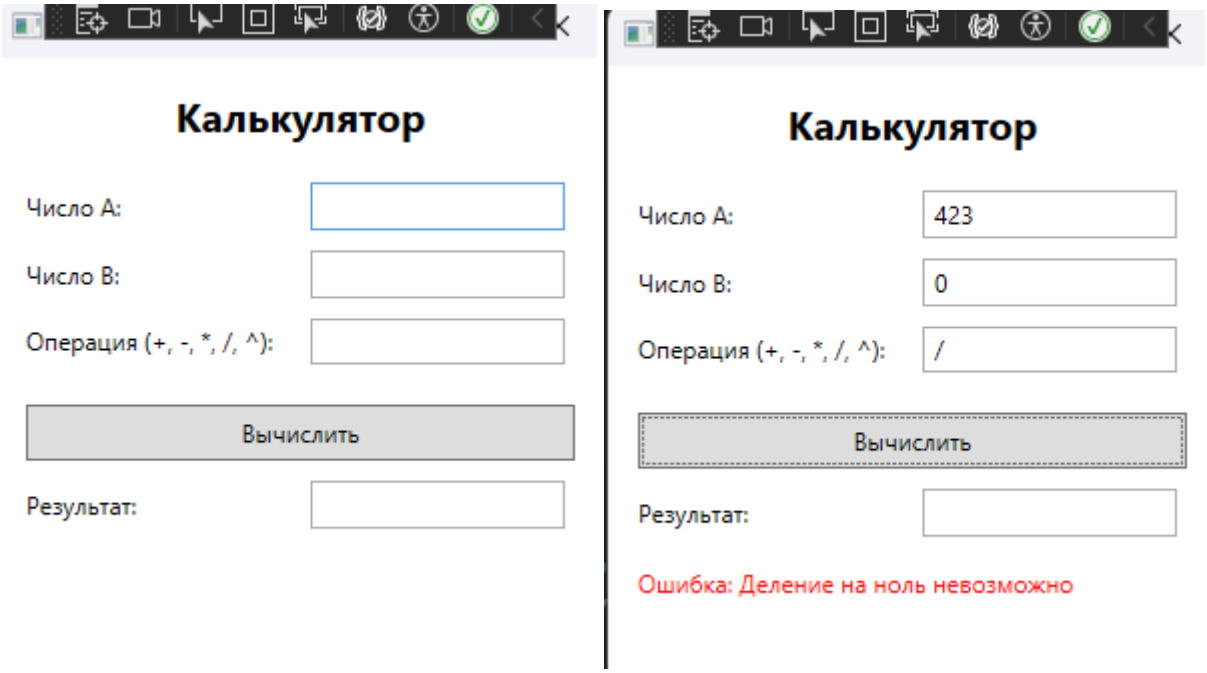
        <TextBlock Grid.Row="0" Grid.ColumnSpan="2" Text="Калькулятор"
                   FontSize="20" FontWeight="Bold"
                   HorizontalAlignment="Center" Margin="0,0,0,15"/>

        <TextBlock Grid.Row="1" Grid.Column="0" Text="Число A:" VerticalAlignment="Center"/>
```

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Работа с приложением интуитивно понятна и состоит из нескольких последовательных шагов. Для начала вычислений пользователю необходимо запустить исполняемый файл приложения. В появившемся окне в первое текстовое поле вводится первый операнд, во второе — второй операнд. В третье поле вводится символ требуемой математической операции: плюс, минус, звездочка, слэш или крышка для возведения в степень. После заполнения полей необходимо нажать кнопку «Вычислить». Если все данные введены корректно, в поле «Результат» отобразится числовой ответ. Если же пользователь допустит ошибку, например, введет буквы вместо цифр или попытается разделить на ноль,

под кнопкой появится красное текстовое сообщение с описанием возникшей проблемы.



ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было успешно разработано приложение «Калькулятор», полностью соответствующее первоначальному техническому заданию. Была спроектирована и реализована модульная архитектура, разделяющая бизнес-логику и пользовательский интерфейс, что является признаком качественного программного обеспечения. Написанный код библиотеки классов покрыт исчерпывающим набором модульных тестов, гарантирующих корректность математических вычислений и надежность обработки исключительных ситуаций. В процессе работы были закреплены практические навыки использования среды Visual Studio, платформы .NET, технологии WPF для создания графических интерфейсов, а также фреймворка MSTest для автоматизированного тестирования кода.